

Restaurant SaaS – System Documentation (SaaS Multi-Tenant Model)

1. Overview

This system is a **cloud-based SaaS solution for restaurants**, providing:

- Contactless QR menus
- Digital ordering (dine-in, takeout, delivery)
- Staff and table management
- Payments and billing
- Analytics and reporting

SaaS Architecture

- **Master DB** – stores global SaaS data (tenants, subscriptions, billing, owner accounts, DB connection info)
- **Tenant DBs** – each restaurant gets its own isolated database for operational data (menus, orders, staff, tables, customers)
- **Dynamic routing** – the system identifies the tenant via subdomain or header, connects to the appropriate tenant DB, and performs business operations

2. Functional Requirements

2.1 Tenant Management

- Create, update, delete tenants (restaurants)
- Subdomain-based identification (`restaurantname.yourapp.com`)
- Database provisioning for each tenant
- Tenant status (trial, active, suspended, expired)
- Global settings per tenant (theme, currency, tax rules)

2.2 Subscriptions

- Plan types: Free, Basic, Pro, Premium
- Limits per plan (menu items, staff, orders/day, categories)
- Billing cycle management (monthly, yearly)
- Payment gateway integration (Stripe, PayPal)
- Plan upgrade/downgrade handling

2.3 Authentication

- **Owner login** (master DB)
- Manages subscriptions, billing, and tenant setup
- **Staff login** (tenant DB)

- Roles: Admin, Manager, Waiter, Chef, Cashier
- Permissions per role

2.4 Menu Management

- Menu categories (Appetizers, Main, Dessert, Drinks, etc.)
- Menu items with name, description, image, price, tax
- Special menus (e.g., lunch menu, happy hour)
- Item availability toggle

2.5 Table Management

- Table setup with number, capacity, section
- QR code assignment per table
- Availability tracking (occupied, free)

2.6 Orders

- Dine-in, takeaway, delivery
- Order status tracking (Received → Preparing → Ready → Served → Paid)
- Order items linked to menu
- Discounts, promotions, and taxes applied per order

2.7 Payments

- Multiple payment methods: Cash, Card, QR Pay, Wallet
- Transaction recording per order
- Invoice/receipt generation

2.8 Customers

- Optional customer accounts for loyalty programs
- History of orders per customer
- Customer feedback

2.9 QR Code System

- Generate QR codes per table or menu section
- QR links open restaurant menu on mobile devices
- Contactless ordering support

2.10 Analytics & Reports

- Sales per day/week/month
- Popular items
- Staff performance reports
- Customer insights

3. Multi-Tenant Architecture

3.1 Master DB

Tables & Purpose:

Table	Description
tenants	Stores restaurant info, subdomain, DB credentials, status
subscriptions	Stores plan, billing cycle, amount, start/end dates
owners	SaaS admin / restaurant owners login credentials
plan_features	Features allowed per plan
global_settings	System-wide settings (tax, default theme, etc.)

Responsibilities:

- Identify tenant from subdomain or request
- Provide DB connection info for dynamic tenant access
- Manage subscription and billing

3.2 Tenant DB

Tables & Purpose:

Table	Description
users	Staff users (Admin, Manager, Waiter, Chef, Cashier)
menu_categories	Menu section categories
menu_items	Items in each category
tables	Table info & QR code mapping
orders	Order header info (customer, table, payment)
order_items	Items in each order
customers	Customer info (optional)
inventory	Stock management
transactions	Payment logs
restaurant_settings	Theme, currency, tax, service charges
qr_codes	Generated QR codes per table/menu

Responsibilities:

- All operational restaurant data
- Staff, orders, menus, tables, payments
- Isolation per restaurant

3.3 Connection Flow

1. Request comes to app → subdomain/header used to identify tenant
2. Query master DB → fetch tenant DB credentials
3. Connect dynamically to tenant DB
4. Execute queries for menu, orders, staff, etc.

4. Security & Isolation

- Tenant databases are **fully isolated**
- No cross-tenant foreign key references
- Staff users only exist in tenant DB
- Passwords hashed (bcrypt/argon2)
- JWT tokens include `tenantId` to scope requests

5. SaaS Advantages

- **Scalable:** Each tenant DB isolated → can scale independently
- **Secure:** Tenant data separation prevents leakage
- **Flexible:** Can add new features without affecting other tenants
- **Backup & Restore:** Per-tenant DB allows selective restore

6. Technical Stack (Recommended)

- **Backend:** Node.js + Express + Drizzle ORM or Prisma
- **Database:** PostgreSQL (master + tenant DBs)
- **Frontend:** React / Next.js
- **Authentication:** JWT tokens
- **Payment:** Stripe / PayPal
- **Hosting:** AWS / GCP / DigitalOcean
- **QR Code:** Dynamic generation via backend API